C PROGRAMMING – ASSIGNMENT 1

TASK 6: ELECTRICITY METER READING VALIDATION

Course Outcome: CO1

Student Name	CH Mukesh reddy
Register Number	192420032
Course / Department	B.Tech – ECE
Compiler / Platform	OnlineGDB – C compiler (GCC-based)
GitHub Repository	https://github.com/<username>/CSA02-<RegisterNumber>-Assignment1

Report prepared from the supplied OnlineGDB execution screenshot and the Assignment 1 submission instructions.

1. Aim

To write and execute a C program that accepts a household number and previous/current electricity meter readings, validates the inputs, and calculates the units consumed when the readings are valid.

2. Problem Analysis

The program collects a household identifier and two meter readings. The household number is stored as an unsigned integer. Meter readings are stored as integers and must be between 0 and 65535 inclusive. The current reading must not be smaller than the previous reading; if it is, the program stops and asks the user to check whether meter rollover occurred. For valid readings, units consumed are calculated as current – previous.

3. Design and Validation Rules

Input / condition	Expected handling
Household number cannot be read as an unsigned integer	Display an invalid household number message and exit with status 1.
Either meter reading cannot be read as an integer	Display an invalid meter readings message and exit with status 1.
A reading is less than 0 or greater than 65535	Display the allowed range and exit with status 1.
Current reading < previous reading	Display a meter-rollover warning and exit with status 1.
Both readings are valid and current ≥ previous	Display household number and units consumed.

4. Algorithm

1. Start the program.
2. Declare variables for household number, previous reading, and current reading.
3. Prompt for and read the household number; stop if input is invalid.
4. Prompt for and read the two meter readings; stop if input is invalid.
5. Check that both readings are within 0–65535.
6. Check that the current reading is not less than the previous reading.
7. Calculate units consumed = current – previous.
8. Print the household number and units consumed, then stop.

5. Pseudocode

```
BEGIN
    INPUT household
    IF household input is invalid THEN
        PRINT invalid household number
        EXIT with error
    ENDIF

    INPUT previous, current
    IF either reading input is invalid THEN
        PRINT invalid meter readings
        EXIT with error
    ENDIF

    IF previous < 0 OR previous > 65535 OR
       current < 0 OR current > 65535 THEN
        PRINT readings must be between 0 and 65535
        EXIT with error
    ENDIF

    IF current < previous THEN
        PRINT check whether meter rollover occurred
        EXIT with error
    ENDIF

    units = current - previous
    PRINT household, units
```

```
END
```

6. C Source Code

The code below is a complete version matching the logic visible in the supplied screenshot. It includes the standard input/output header and main function so it can be compiled independently.

```
#include <stdio.h>

int main(void)
{
    unsigned int household;
    int previous, current;

    printf("Enter household number: ");
    if (scanf("%u", &household) != 1)
    {
        printf("Invalid household number.\n");
        return 1;
    }

    printf("Enter previous and current readings: ");
    if (scanf("%d %d", &previous, &current) != 2)
    {
        printf("Invalid meter readings.\n");
        return 1;
    }

    if (previous < 0 || previous > 65535 ||
        current < 0 || current > 65535)
    {
        printf("Readings must be between 0 and 65535.\n");
        return 1;
    }

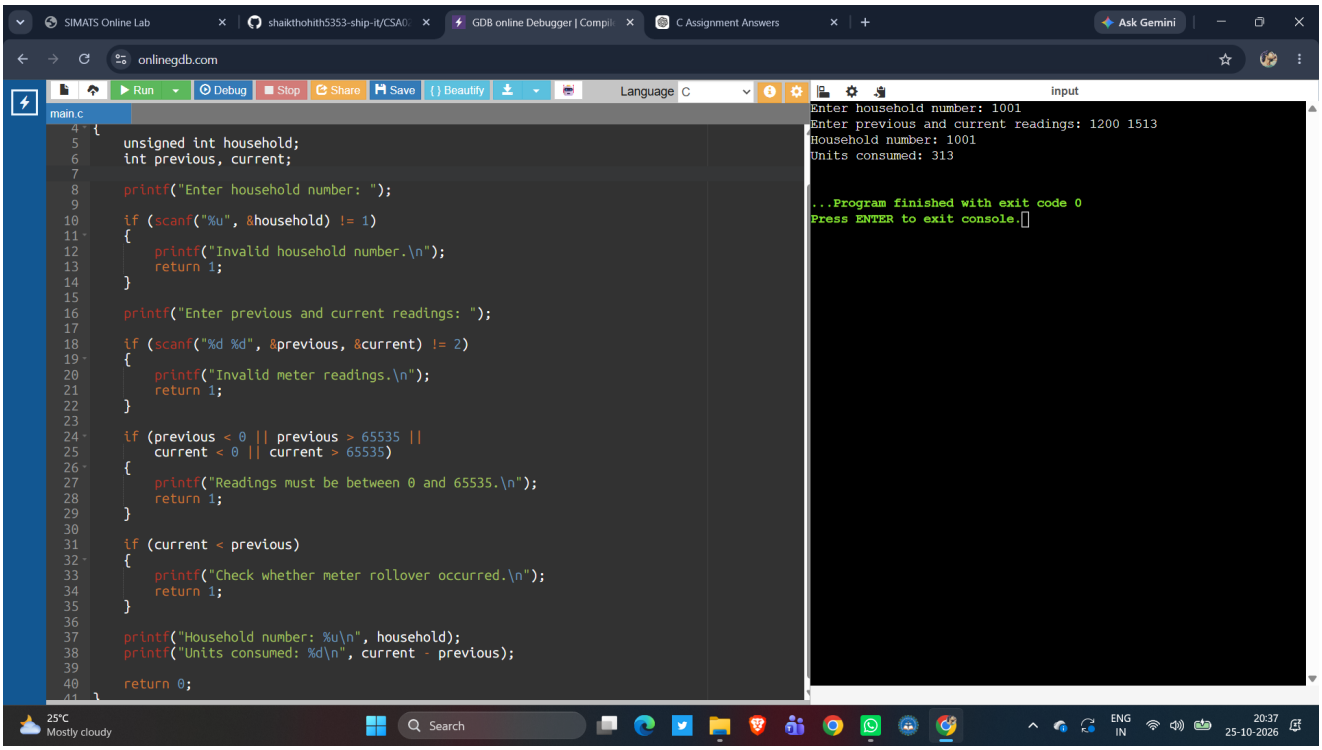
    if (current < previous)
    {
        printf("Check whether meter rollover occurred.\n");
        return 1;
    }

    printf("Household number: %u\\n", household);
    printf("Units consumed: %d\\n", current - previous);

    return 0;
}
```

7. Execution Evidence (Supplied Screenshot)

The screenshot shows a successful OnlineGDB run with household number 1001, previous reading 1200, and current reading 1513. The program reports 313 units consumed, which is correct because 1513 – 1200 = 313.




```c
{
    unsigned int household;
    int previous, current;

    printf("Enter household number: ");

    if (scanf("%u", &household) != 1)
    {
        printf("Invalid household number.\n");
        return 1;
    }

    printf("Enter previous and current readings: ");

    if (scanf("%d %d", &previous, &current) != 2)
    {
        printf("Invalid meter readings.\n");
        return 1;
    }

    if (previous < 0 || previous > 65535 ||
        current < 0 || current > 65535)
    {
        printf("Readings must be between 0 and 65535.\n");
        return 1;
    }

    if (current < previous)
    {
        printf("Check whether meter rollover occurred.\n");
        return 1;
    }

    printf("Household number: %u\n", household);
    printf("Units consumed: %d\n", current - previous);

    return 0;
}
```

8. Test Cases and Results

Test	Inputs	Expected result	Observed / predicted result
TC1 – Normal	Household 1001; 1200 1513	Household number: 1001; Units consumed: 313	Matches supplied screenshot.
TC2 – Equal readings	Household 1002; 500 500	Units consumed: 0	Predicted from code; execute to verify.
TC3 – Lower current	Household 1003; 900 700	Rollover warning; exit with error	Predicted from code; execute to verify.
TC4 – Upper boundary	Household 1004; 0 65535	Units consumed: 65535	Predicted from code; execute to verify.
TC5 – Out of range	Household 1005; 10 65536	Readings range error	Predicted from code; execute to verify.
TC6 – Invalid reading text	Household 1006; 100 abc	Invalid meter readings	Predicted from code; execute to verify.

Note: Only TC1 is evidenced by the supplied screenshot. TC2–TC6 are recommended test cases and their results are predictions based on the code; run them in OnlineGDB and replace the status with screenshots before final submission.

9. Complexity Analysis

Time complexity: O(1), because the program performs a fixed number of input, validation, arithmetic, and output operations regardless of input size. Space complexity: O(1), because it stores only three scalar variables and uses no input-sized data structures.

10. Result and Conclusion

The program successfully validates household and meter-reading inputs and calculates electricity units consumed for valid readings. For the supplied test case, the calculation is 1513 – 1200 = 313 units, matching the displayed output. The program intentionally rejects a current reading lower than the previous reading rather than calculating rollover automatically; this is a limitation to document if rollover support is required by a later design challenge.

11. Submission Checklist

- Save the source as main.c inside Task06.
- Save the supplied execution screenshot inside Task06.
- Add any required input files (none are needed for this interactive program).
- Add the exact compiler/version and compile command to README.md after confirming the environment used.
- Replace the GitHub placeholder on the cover page with the working repository URL.
- Commit Task06 separately and execute the remaining test cases before submitting.

Suggested compile command: gcc -std=c11 -Wall -Wextra -pedantic main.c -o task06

Suggested run command: ./task06 (Linux/macOS) or task06.exe (Windows, depending on the compiler environment).